Individual Project (CS3IP16)

Department of Computer Science
University of Reading

Project Initiation Document

PID Sign-Off

Student No.	**32001784**
Student Name	**Zain Chohan**
Email	**dt001784@student.reading.ac.uk**
Degree programme (BSc CS/BSc CSwIY)	**BSc CS**
Supervisor Name *(Consultation with supervisor is mandatory)*	**Richard Mitchell**
	Supervisor to sign PID form on Bb (grade centre)
Date	**09/10/2025**

SECTION 1 – General Information

Project Identification

1.1	**Project Title**
	E-Learning Web Pages for Computer Science
1.2	**Please describe the project with key-phrases (max 5)**
	• CPU Pipeline Visualization • Interactive Learning Tool • Hazard Detection and Resolution • Instruction-Level Parallelism • Computer Architecture Education
1.3	**E-logbook maintenance agreed with supervisor** *Use Google doc, OneDrive, or any mobile App whereby you will be able to generate a PDF copy*
	https://docs.google.com/document/d/19doY5bvA07alLo5Tr-uBe4qYTx9TkV4--4xw6d_BXac/edit?usp=sharing
1.4	**GitLab link for maintain source code and research data** *Any change in GitLab link and Source code repository MUST be explicitly mention in final report*
	https://csgitlab.reading.ac.uk/dt001784/cs3ip_zain_32001784

SECTION 2 – Project Description

2.1	**Summarise the project's background in terms of research field /application domain (max 100 words).**
	Understanding CPU instruction pipelining is fundamental to computer architecture education, yet students often struggle with abstract concepts like hazards, stalls, and forwarding. Traditional textbooks use static diagrams that fail to convey the dynamic nature of pipeline execution. Interactive e-learning tools have proven effective in teaching complex computational concepts by allowing students to visualize and experiment with real-time simulations. This project addresses the need for an interactive web-based pipeline simulator that demonstrates how modern processors achieve instruction-level parallelism while handling various hazards and optimizations.
2.2	**Summarise the project aims, objectives and outputs (max 250 words).** These aims, objectives, and outputs should appear as the tasks, milestones and deliverables in your project plan (fill out Section 3).

	Aims: The project aims to create an interactive web-based CPU pipeline simulator that helps students understand instruction pipelining, hazard detection, and resolution techniques through visual demonstration and hands-on experimentation. Objectives: ● Develop a visual representation of the 5-stage pipeline (Fetch, Decode, Execute, Memory, Write-back) showing instruction flow in real-time ● Implement detection and visualization of three hazard types: data hazards (RAW, WAR, WAW), control hazards (branch misprediction), and structural hazards ● Demonstrate hazard resolution techniques including pipeline stalling, data forwarding/bypassing, and branch prediction ● Create an interactive interface allowing users to input assembly instructions and step through execution ● Display CPU state including register values, memory contents, and performance metrics (CPI, cycles, throughput) ● Provide pre-loaded example programs demonstrating different hazard scenarios ● Include educational content explaining pipeline concepts and hazard mitigation strategies Outputs: ● A fully functional web-based pipeline simulator accessible via modern browsers ● Interactive visualization showing instruction progression through pipeline stages ● Real-time hazard detection and highlighting with explanatory tooltips ● Editable instruction input with assembly language syntax support ● Performance analysis dashboard showing efficiency metrics ● Educational tutorial mode with guided examples ● Comprehensive documentation for users and future developers ● Test suite validating simulator accuracy against known pipeline behaviors
2.3	**Initial project specification – roughly indicate key features and functions of your finished program/application. Indicate possible method, data source, technology etc. (max 400 words)** (Sensible and relevant Charts, Table, and Figures can be used)

Core Features:

1. Pipeline Visualization Engine

- Five horizontal stages displayed as a grid showing instruction progression
- Color-coded instruction blocks moving through stages cycle-by-cycle
- Clock cycle counter with play/pause/step controls
- Adjustable simulation speed (slow motion to fast forward)

2. Instruction Input System

- Text editor for MIPS-style assembly code input
- Syntax highlighting and basic error checking
- Pre-loaded example programs (loops with dependencies, branches, memory operations)
- Instruction set includes: arithmetic (ADD, SUB, MUL), memory (LW, SW), branches (BEQ, BNE, J), and logical operations (AND, OR)

3. Hazard Detection & Visualization

- Data hazards: highlight dependent instructions and show RAW dependencies with connecting lines
- Control hazards: show branch prediction (predict-not-taken) and misprediction penalties
- Structural hazards: demonstrate resource conflicts when applicable
- Visual indicators (warning icons, color changes) when hazards occur
- Pop-up explanations describing why each hazard occurred

4. Hazard Resolution Mechanisms

- Pipeline stalling: show NOP bubbles inserted into pipeline
- Data forwarding: display forwarding paths between pipeline stages with arrows
- Branch prediction: toggle between different strategies (always not-taken, always taken, simple predictor)
- Statistics showing stall cycles, forwarded values, and branch prediction accuracy

5. CPU State Display

- Register file showing all 32 MIPS registers with real-time value updates
- Memory viewer displaying relevant data memory locations
- Program counter (PC) tracking
- Performance metrics: total cycles, instructions completed, CPI calculation, pipeline utilization percentage

6. Educational Components

- Tutorial mode with step-by-step explanations
- Quiz mode: present hazard scenarios and ask users to predict outcomes
- Side panel with contextual help explaining current pipeline state
- Comparison view showing pipelined vs non-pipelined execution time

Technology Stack:

- Frontend: HTML5, CSS3, JavaScript (vanilla or React for component management)
- Visualization: Canvas API or SVG for pipeline graphics, possibly D3.js for data-driven diagrams
- Code Editor: CodeMirror or Monaco Editor for syntax highlighting

	• Testing: Jest for unit testing pipeline logic • Deployment: Static hosting (GitHub Pages, Netlify, or university servers) Data Sources: • Standard MIPS instruction set architecture documentation • Computer architecture textbooks (Patterson & Hennessy) for validation examples • Example assembly programs demonstrating various hazard scenarios Development Method: • Iterative development: start with basic pipeline, add hazards progressively • Test-driven development for instruction execution accuracy • User testing with CS students for usability feedback
2.4	**Describe the social, legal and ethical issues that apply to your project. Does your project require ethical approval? (If your project requires a questionnaire/interview for conducting research and/or collecting data, you will need to apply for an ethical approval)**
	Ethical Approval: This project involves the development of an educational e-learning system and does not collect any personal or sensitive user data as part of its normal operation. However, a short anonymous questionnaire will be conducted with a small group of Computer Science students to evaluate the educational effectiveness and usability of the tool. This questionnaire will form part of the Student Learning Experience Assessment (SLEA) and will follow University ethical procedures. No personally identifiable information will be collected, and participation will be entirely voluntary. An ethical approval application will therefore be submitted before the questionnaire is distributed. Social Issues: • Accessibility: The tool will be designed to follow WCAG accessibility guidelines, including support for keyboard navigation, screen readers, and color-blind-friendly palettes. • Digital Divide: As a web-based tool, it requires modern browsers and internet access, which may disadvantage users with limited resources. An offline mode could be considered for inclusivity. • Educational Equity: The tool will be freely available to ensure all students can benefit regardless of financial background. Legal Issues: • Intellectual Property: All code will be original or will use open-source libraries with appropriate licenses (MIT, Apache 2.0), with proper attribution provided. • Copyright: Educational content and explanations will be written originally or derived from public-domain or openly licensed sources. • Software Licensing: The final project will use an open-source license (e.g., MIT) to allow educational reuse. • MIPS Architecture: The MIPS instruction set will be used strictly for educational demonstration under fair use; no proprietary implementations will be replicated. Data Protection: • No personal data will be stored or processed by the system itself. • Questionnaire responses will be collected anonymously and stored securely in accordance with GDPR and University data-handling policies. • If future analytics are added, they will be limited to anonymous, aggregated data only. • No cookies or tracking mechanisms will be used beyond those essential for basic functionality. Academic Integrity: • All external resources and references will be appropriately cited. • Documentation will emphasize the educational intent of the tool to discourage misuse for assignment completion without understanding.

2.5	**Identify the items you may need to purchase for your project. A cost upto £200 can be applied (include VAT and shipping if known). You need to have the consent of your supervisor. Your request will be assessed by the department.**
	No purchases required. This project can be completed entirely with free and open-source software.
2.6	**State whether you need access to specific resources within the department or the University e.g. special devices and workshop**
	Required Resources: ● Standard laboratory computers with modern web browsers (Chrome, Firefox, Edge) for development and testing ● University web server space for hosting the final application (if not using GitHub Pages) ● Access to the department's GitLab (already arranged - link provided in Section 1.4)

SECTION 3 - Project Plan
Please provide your project plan. Below is an example project plan, you can use any tool or software to generate yours.
START DATE: 06/10/25

Project Stage / Task	W2-4 Oct 6-24	W4-6 Oct 27-Nov 7	W6-8 Nov 10-21	W8-10 Nov 24-Dec 5	W10-12 Dec 8-19	W12-14 Dec 22-Jan 9	W14-18 Jan 12-Feb 6	W18-22 Feb 9-Mar 6	W22-26 Mar 9-Apr 3	W26-31 Apr 6-May 1
1. Background Research & Analysis										
1.1 Literature review - CPU architecture	W2-4									
1.2 Review existing pipeline simulators	W2-4									
1.3 Technology stack evaluation		W4-6								
1.4 Requirements specification		W4-6								
1.5 Literature Review & SLEA Evaluation										
1.5a Draft literature review				W8-10	W10-12					
1.5b Design & deploy SLEA questionnaire			W6-8							
1.5c Collect & analyze questionnaire data				W8-10						
2. System Design										
2.1 System architecture & UI/UX design		W4-6								
2.2 Pipeline algorithms & data structures		W4-6								
3. Implementation										
3.1 Core pipeline engine			W6-8							
3.2 Hazard detection & resolution				W8-10						
3.3 Visualization & UI components					W10-12					
3.4 Educational content & tutorials						W12-14				
4. Testing & Evaluation										
4.1 Unit & integration testing							W14-18			
4.2 User acceptance testing & feedback								W18-22		
4.3 Bug fixes & refinement								W18-22		
5. Documentation & Final Report										
5.1 Technical documentation								W18-22		
5.2 Final report draft									W22-26	
5.3 Final report & poster completion										W26-31
6. Key Milestones & Deliverables										
M1: PID Submission (Oct 6-10, 2025)	W2-4									
M2: Literature Review (Dec 15-19, 2025)					Dec 15-19					
M3: Design Complete (Early Jan 2026)						Early Jan				
M4: Feedback Form (Feb 18-20, 2026)								Feb 18-20		
M5: Working Prototype (Early Feb 2026)							Early Feb			
M6: Work Demonstration (Mar 2-6, 2026)								Mar 2-6		
M7: Testing Complete (Early Mar 2026)								Early Mar		
M8: Poster Submission (Apr 20-22, 2026)										Apr 20-22
M9: Final Submission (May 1, 2026)										May 1

Legend

Research & Literature Review · Design Phase · Implementation · Testing & Evaluation · Documentation & Reporting · Key Milestones